Intermediate (wrap real logic behind a route) — Hints

> Back to the exercise · Hint 1 · Hint 2 · Solution

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint builds toward the full solution.

- Hint 1 — The idea, and the exact pieces
- Hint 2 — The plan, and almost the whole thing

Hint 1 — The idea, and the exact pieces

Declaring `def chat(request: ChatRequest):` is the whole trick — FastAPI reads that type hint, and before your function body runs even once, it parses the incoming request body as JSON, checks it against `ChatRequest`, and automatically replies with a 422 if it doesn't match. You never write that checking code yourself; it's the same idea as Doc04/06's tool-argument Pydantic models, just applied to an HTTP body.

Doc02's error material comes back here in a new form: a bad-input problem is the *client's* fault (4xx — don't blindly retry the same request), an unexpected internal failure is the *server's* fault (5xx). `HTTPException` stops the route immediately and sends that exact status code and detail message back.

Two things a naive `except Exception: raise HTTPException(500, detail=str(exc))` gets wrong: it treats *every* failure as a 500 (even ones that are really the client's fault, like an empty message), and it puts raw exception text — which can include file paths or even a stray API key — straight into the client's reply. So the real design question is: **what deserves a 500 versus a 4xx, and what should the client ever see about *why*?**

The exact pieces:

- `class ChatResponse(BaseModel): reply: str` — the output shape, same pattern as `HealthResponse` in the previous exercise.
- A custom exception, `class ChatInputError(Exception): pass`, raised on purpose for bad-but-well-formed input (an empty message) — caught separately and mapped to 400, not 500.
- Moving `ChatRequest` / `ChatResponse` into their own file (`practice/chat_schemas.py`) — the same split as the Build Task's `schemas.py` .

- `logger.exception("chat route failed")` inside the `except Exception` branch — logs the full traceback on the server, while the client still gets only a generic message. Get the logger from Doc01's `get_logger(__name__)` (copy `logging_setup.py` into `practice/`, unchanged).
- Never put `str(exc)` into an `HTTPException(detail=...)` for the 500 case — that's exactly the leak this exercise tests for.
- A **request ID**: a short random string made once per request (`str(uuid.uuid4())[:8]`) in a middleware (`@app.middleware("http")`), saved on `request.state.request_id` , sent back in an `X-Request-ID` header, and put at the start of every log line for that request — so you can find every line from one failed call, even under busy traffic.

Hint 2 — The plan, and almost the whole thing

```
define ChatRequest, ChatResponse, ChatInputError

logger = get_logger(__name__)          # Doc01

function run_chat(message) -> str:
    if message is empty: raise ChatInputError
    if message is "boom": raise RuntimeError   # a fake crash, on command
    return reply text

middleware: make a short request_id, save it on request.state,
            add it to the X-Request-ID response header

route POST /chat:
    try:
        return ChatResponse(reply=run_chat(request.message))
    except ChatInputError as exc:
        raise HTTPException(400, detail=str(exc))
    except Exception:
        logger.exception("[request_id] chat route failed")  # server only
        raise HTTPException(500, detail="Something went wrong.")
```

Here's most of it — wire up the logger call yourself:

```
# practice/chat_route_practice.py
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel

from logging_setup import get_logger   # Doc01 Build Task

logger = get_logger(__name__)
```

```
app = FastAPI()

class ChatRequest(BaseModel):
    message: str

class ChatResponse(BaseModel):
    reply: str

class ChatInputError(Exception):
    pass

def run_chat(message: str) -> str:
    if not message.strip():
        raise ChatInputError("message cannot be empty")
    if message == "boom":
        raise RuntimeError("simulated internal failure")
    return "You said: " + message

@app.post("/chat", response_model=ChatResponse)
def chat(request: ChatRequest) -> ChatResponse:
    try:
        reply_text = run_chat(request.message)
        return ChatResponse(reply=reply_text)
    except ChatInputError as exc:
        raise HTTPException(status_code=400, detail=str(exc))
    except Exception:
        # your turn: log the real error with logger.exception(...),
        # then raise a 500 with a generic message only
        ...
```

Send `{"message": ""}` and confirm a 400. Send `{"message": "boom"}` and confirm: the client gets a plain 500 message, and the *real* `RuntimeError: simulated internal failure` traceback only shows up in your terminal. Then add the request-ID middleware — the Solution's Approach 4 shows it.

Full solution: Show me the solution